

### Structure

|        |                                                |
|--------|------------------------------------------------|
| 11.0   | Introduction                                   |
| 11.1   | Objectives                                     |
| 11.2   | Brief Introduction to Binary Search Tree (BST) |
| 11.3   | Scapegoat trees                                |
| 11.3.1 | Operations on Scapegoat tries                  |
| 11.4   | Tries                                          |
| 11.4.1 | Binary Tries                                   |
| 11.5   | X-fast Tries                                   |
| 11.5.1 | Operations on X-fast tries                     |
| 11.6   | Y-Fast Tries                                   |
| 11.6.1 | Operations on Y-fast tries                     |
| 11.7   | Summary                                        |
| 11.8   | Solutions/Answers                              |
| 11.9   | Further Readings                               |

---

## 11.0 INTRODUCTION

---

In this unit we discuss various kinds of trie data structure. For efficient searching the data we use the trie. We shall closely look at various operations such as search, insert, delete, successor and predecessor on the trie data structure. We also look at optimizing the time complexity in the trie data structure.

---

## 11.1 OBJECTIVES

---

After going through this unit, you should be able to

- understand key concepts of scapegoat tries,
- understand tries and binary tries,
- understand key concepts of x-fast tries and y-fast tries

---

## 11.2 BRIEF INTRODUCTION TO BINARY SEARCH TREE (BST)

---

In this unit we discuss the Tries that are type of binary search trees. So, we briefly discuss the binary search tree. Binary search tree is a sorted tree that stores the keys in ordered way. For any given node in the binary search tree, the keys in left side is lesser than keys in the right side.

As the binary search tree is sorted, the data structure is useful for searching and sorting the data. To search for a key in the BST, we start from root node and if the search key is less than root node we check for left node else we check the right node. We recursively search for the key till we exhaust the nodes. To find the successor of the key  $k$ , we find the smallest key that is greater than  $k$  and for finding the predecessor for key  $k$ , we find the largest key that is smaller than  $k$ . The complexity of BST for search, insert and delete is  $O(n)$  where  $n$  is the number of nodes in the tree.

---

## 11.3 SCAPEGOAT TRIES

---

The scapegoat tries are self-balancing binary search tree that provides the worst-case performance of  $O(\log n)$  for search and  $O(\log n)$  amortized time for insert and delete operation.

We say that the binary search tree is balanced when the weight of the left nodes matches the weight of the right nodes. We define the  $\alpha$ -weight-balanced tree by the below-given equation:

$$\text{Size}(\text{left}[x]) \leq \alpha * \text{size}(x) \text{ and} \\ \text{Size}(\text{right}[x]) \leq \alpha * \text{size}(x)$$

Where

- $0.5 \leq \alpha < 1$
- $X$  is the node of the tree and  $\text{left}(x)$  is the left child of  $x$  and  $\text{right}(x)$  is the right child of  $x$ .
- The  $\text{size}(x)$  is defined as number of keys stored in the sub-tree of  $x$  including the key stored at  $x$ .

Figure 1 provides the scapegoat tree that is  $\alpha$ -weight-balanced

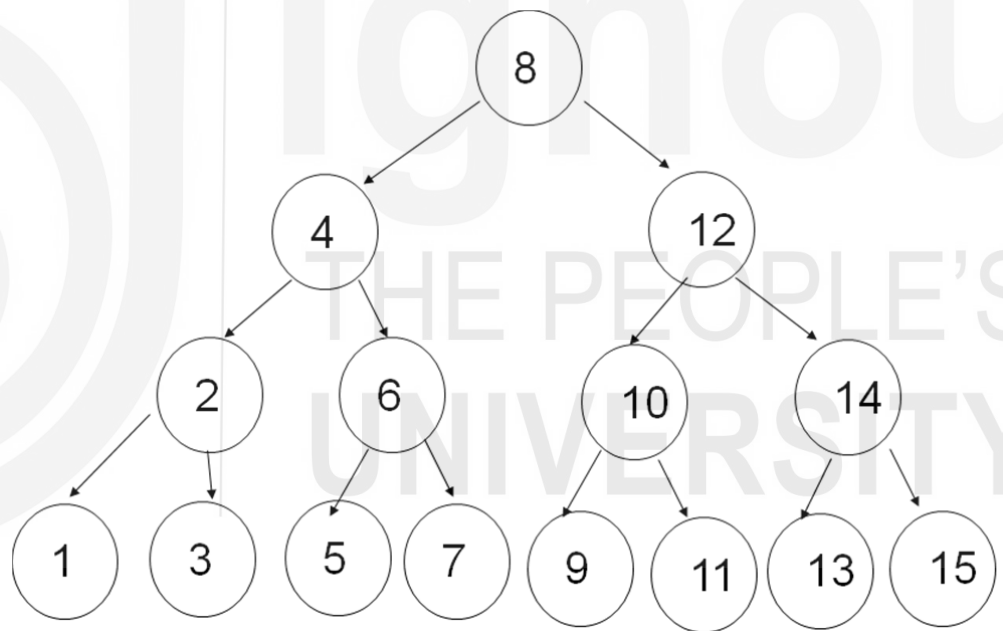


Figure 1 Sample Scapegoat tree

### 11.3.1 Operations on Scapegoat tries

In this section we define various operations on Scapegoat tries.

#### Search Operation

The search operation is similar to that of the binary search tree.

#### Insert Operation

For inserting a new node into the tree, we will find a scapegoat node. Given below are the steps to find the scape goat node for inserting a new value  $Y$  into the tree:

1. Create the new node  $x$  and insert in natural way

2. Walk up the tree hierarchy to find the scapegoat node that is not  $\alpha$ -weight-balanced. If  $x_i$  is the node that is not  $\alpha$ -weight-balanced, we compute the size of as follows:

$$\text{Size}(X_{i+1}) = \text{Size}(X_i) + \text{size}(\text{brother}(X_i)) + 1$$

Where  $\text{brother}(x_i)$  is the other child of  $X_i$ 's parent or null if another child does not exist

Let us look at an insert example where we try to insert element 4.5 into a tree as depicted in Figure 2.

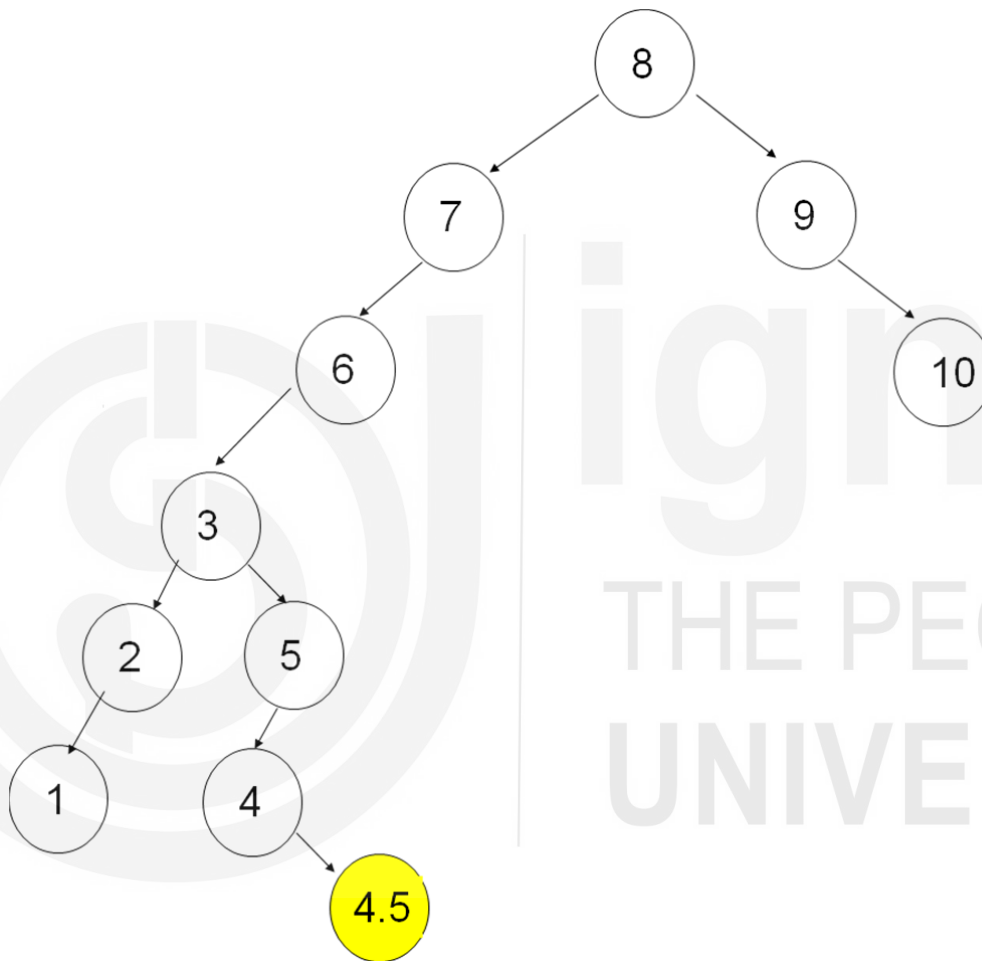


Figure 2 Inserting new node to Scapegoat tree

If we traverse the tree to find the scapegoat node that is not load balanced we reach, node with key 6. So, we balance the tree at node 6. The balanced tree is depicted in Figure 3.

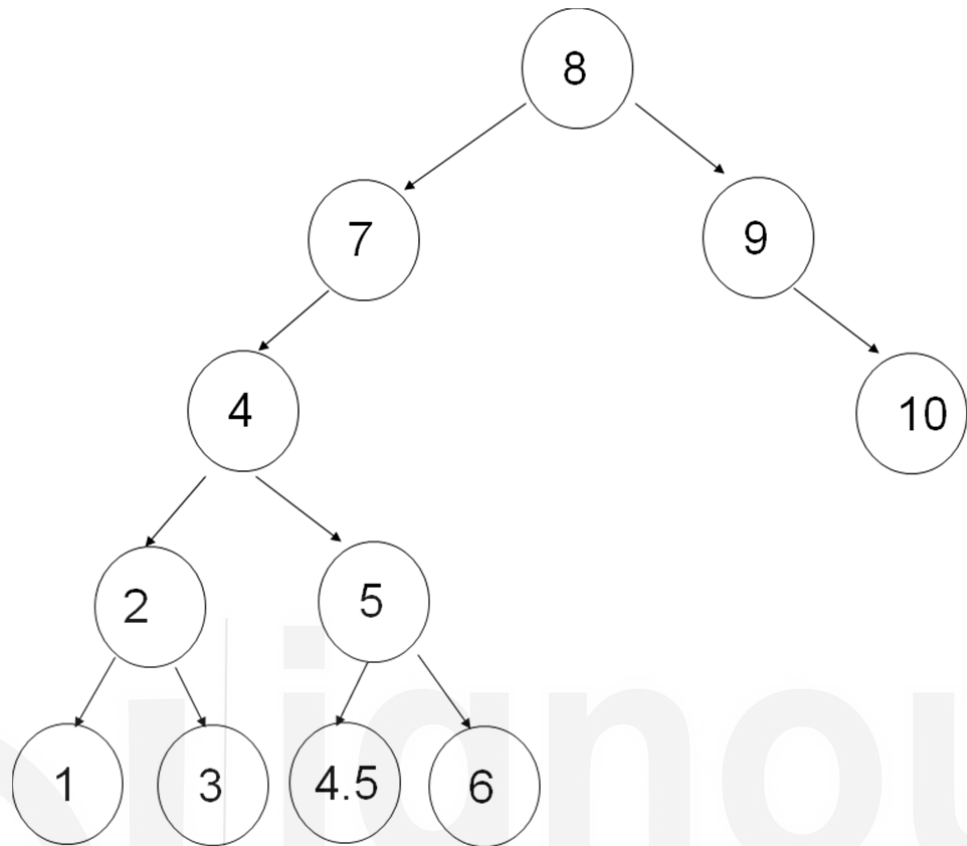


Figure 3 New scapegoat tree rebalanced after inserting

**Delete operation**

Given below are the steps for deleting the node x:

1. If the node x is a leaf node, we can delete it.
2. If the node x has single left child or single right child, we replace x with its only child.
3. If the node x has both left child and right child, we find the lowest value node in the right sub-tree and replace x with it.

**☛ Check Your Progress – 1**

1. The scapegoat tries are \_\_\_\_\_ binary search tree.
2. For inserting the node we find the \_\_\_\_\_ node
3. The worst-case performance of scapegoat search operation is \_\_\_\_\_
4. In the binary search tree, the left node value is \_\_\_\_\_ than right node value
5. The complexity of BST for search, insert and delete is \_\_\_\_\_

---

## 11.4 TRIES

---

Tries are tree-based data structure that are used to search for string-based keys in a given set. We use individual characters of a string as edges that connect the nodes of the tree. Given below are the main characteristics of tries:

1. The child nodes of a node are associated with a common prefix.
2. The key is denoted by the position of the node
3. String search can be done by using the depth-first search.

There are many real-world use cases that involve integer-based information retrieval. For instance, id number-based information retrieval, account number-based search all need integer-based lookup. For such use cases we can use bitwise tries for faster information retrieval. Tries are also used in web search engines, for text search use cases.

The average time complexity for search, insert and delete operations for trie data structure is  $O(n)$ .

Figure 4 depicts a sample Trie

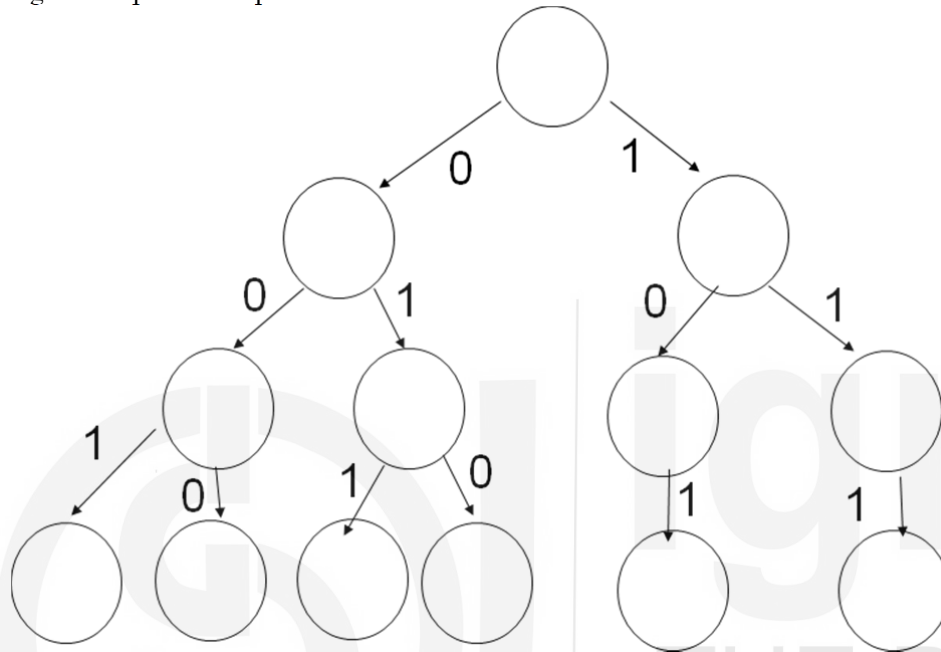


Figure 4 Sample Trie

### 11.4.1 Binary tries

The binary tries use bits as keys that can be used to represent integer. We can use the bit representation of individual characters of a string which can then be used to traverse the binary Tries. In the binary trie only the leaf node holds the key. We compare the bits of the tree to find the key.

#### Search operation in Binary trie

Given below are steps for searching for a key in binary trie:

1. Start the search with root node
2. Initialize  $i=0$  and compare the  $i^{\text{th}}$  bit of key.
3. If the key bit value is 0 check the left node, else search the right node.
4. If the current node is leaf and if all the bits of the key match the path, return success else return failure
5. If the current node is not leaf increment the counter  $i$  and repeat the steps 3 through 5.

#### Insert operation in Binary Trie

We insert the keys based on their bit values. The initial 0 bit will go to left and 1 bit to the right. We follow the process till all the bits in all keys are inserted.

Given below are the steps for insert operation:

1. Search for key  $k$  in the tree.

2. When the search failure happens on a non-leaf node, we simply link the non-leaf node to the new node
3. If the search failure happens on leaf node, we create a new internal node to accommodate the new key and existing key.

### Delete operation in Binary Trie

Given below are the steps for deleting the node x:

1. Search for node x in the tree. When the leaf node x is reached, it is deleted.
2. We traverse back in the search path for x deleting the nodes until we reach a node u that has child not in the search path for x.
3. We traverse back to the root node deleting the descendent pointer to node x.

Figure 5 is an example of inserting the following keys:

S 01001  
M 11001  
U 00110

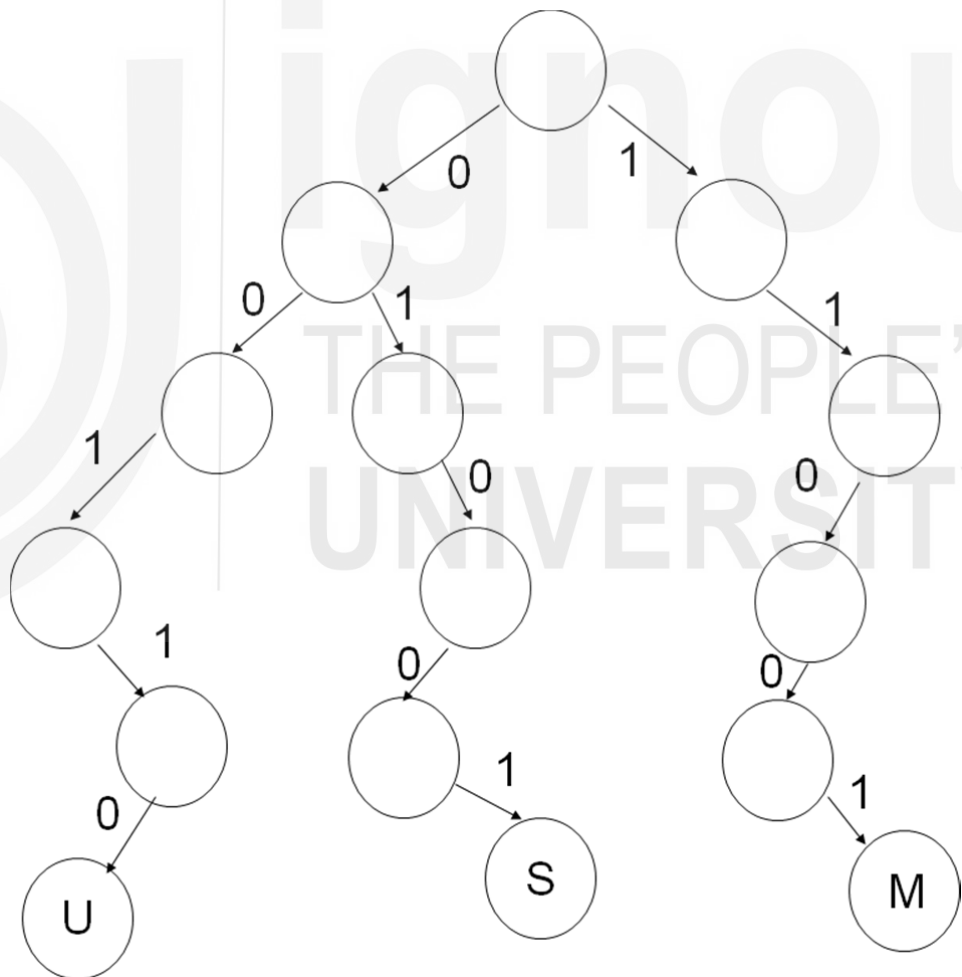


Figure 5 Binary Trie with keys inserted

As we can notice in Figure 5, the keys in left subtree are smaller when compared to keys in the right subtree for any given node.

---

## 11.5 X-FAST TRIES

---

X-fast tries are bitwise tries that stores the integers from a bounded domain. The complexity of successor operation is  $O(\log \log M)$  where  $M$  is the domain's maximum value; find operation is  $O(1)$ ; insert and delete operation is  $O(\log M)$  where  $M$  is the domain's maximum value. The subtree uses a common prefix from its parent node. The leaf node represents values from 0 to  $M-1$  where  $M$  is the domain's maximum value.

The left child adds 0 to the prefix and the right child adds 1 to the prefix.

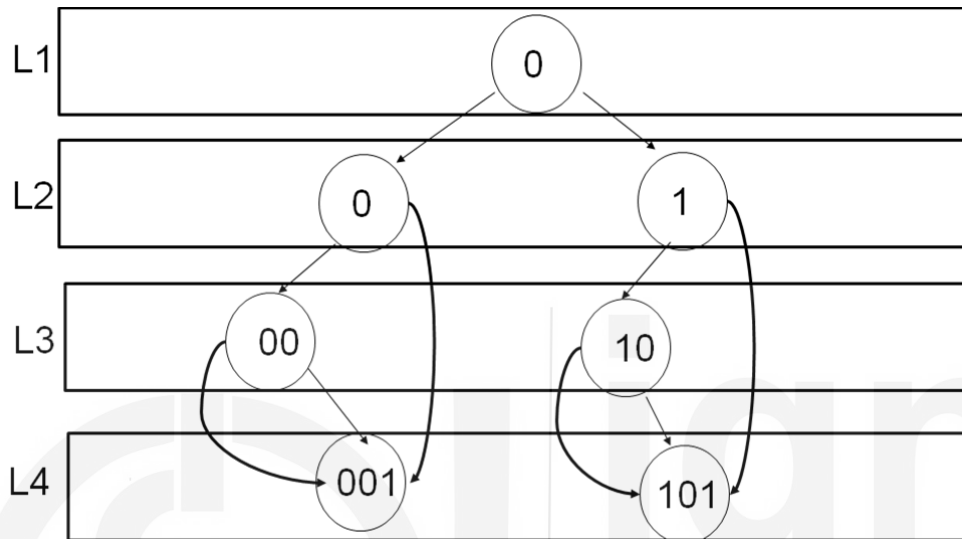


Figure 6 X-fast trie

We have depicted a four-level x-fast trie in Figure 5. Each level of the x-fast trie is implemented as hash table. The internal nodes are represented only if their subtree has one or more leaf nodes. If the internal node does not have a left child, then the left pointer points to the smallest leaf node in the right subtree which is called descendant pointer.

The tree depicted in Figure 5 has domain size of 8 from 0 to 7. We store the nodes 0 (001) and 5 (101).

### 11.5.1 Operations on X-fast tries

In this section we shall look at the main operations on X-fast tries

#### Find operation

For finding value of key  $k$ , we can look up the hash table indexing the leaf nodes. This is a constant time operation.

#### Successor and Predecessor

To find the successor of key  $k$ , we follow these steps:

1. Using the prefix of key, we perform the binary search by querying the hash table to find the lowest ancestor of  $k$ .
2. Once we get a match we use the descendant pointer of  $k$
3. For the predecessor use the previous leaf and for successor use the next leaf.

**Insert Operation**

To insert key  $k$  with value  $v$ , we follow these steps:

1. Find the predecessor and successor of  $k$
2. Create a new leaf node between predecessor and successor of  $k$  and point it to  $v$ .
3. Start traversing the tree from the root to leaf and create the missing internal nodes and adding the descendant pointers wherever necessary.

**Delete Operation**

To delete the key  $k$ , we follow these steps:

1. Remove the  $k$  from the hash table indexing the leaf nodes.
2. Delete the  $k$  node and link its predecessor and successor.
3. Traverse the tree from the root and delete the internal nodes that have  $k$  as only nodes within its subtrees and update the descendant pointers appropriately.

---

## 11.6 Y-FAST TRIES

---

Y-fast tries are bitwise tries that store integers from bounded context. They are the improved version of X-fast tries that optimize the memory. Similar to the X-fast tries, all subtrees will have a common prefix of its parent.

The complexity of successor operation is  $O(\log \log M)$  where  $M$  is the domain's maximum value; find operation is  $O(1)$ ; insert and delete operation is  $O(\log M)$  where  $M$  is the domain's maximum value.

The Y-fast trie consists of two data structures – X-fast trie at the top and the balanced binary search tree in the bottom. We have depicted the Y-fast Trie in Figure 6.

We chose a representative  $r$  from the binary search tree and store it in the X-fast trie. The  $r$  value should be smaller than its successor in the X-fast trie.

One of the main differences between X-fast trie and Y-fast trie is that in the Y-fast trie, the X-fast trie operates on the representatives of the binary search tree. The leaf node point to the binary search tree.

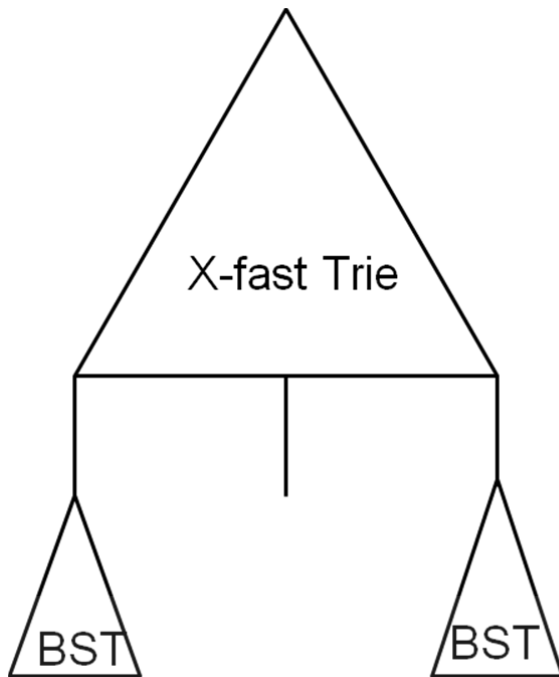


Figure 7 Y-fast Trie

### 11.6.1 Operations on Y-fast tries

In this section we shall look at the main operations on Y-fast tries.

#### Find operation

Given below are the steps to find the key  $k$ :

1. Find the successor and predecessor of  $k$  in the X-fast trie. We can do this by finding the smallest representative  $r$  that is greater than  $k$  and largest representative  $r^1$  that is smaller than  $k$  in the X-fast trie.
2. We carry out the binary search for  $k$  in the binary search trees represented by  $r$  and  $r^1$ .

#### Successor and Predecessor

1. To find successor of  $k$ , search for smallest representative  $r$  in the X-fast trie.
2. Find the predecessor of  $r$  in X-fast trie.
3. Search for the binary trees pointed by these two representatives for getting the successor of  $k$ .

#### Insert Operation

To insert the key  $k$ , given below are the steps:

1. Insert the key to the tree that has the successor of  $k$ .
2. Point the node to  $v$
3. If the tree contains more than  $2 \log U$  elements (where  $U$  is the maximum value of the domain), then remove it representative from the X-fast trie. Split the binary search tree into two and add the representatives from each of the binary search tree into the X-fast trie.

#### Delete operation

Given below are the steps for deleting the key  $k$ :

1. Delete the key  $k$  from the tree

2. If the elements in the tree drop below  $(\log U)/4$  (where  $U$  is the maximum value of the domain), merge the tree with its predecessor or successor. Remove the representatives of the merged trees from the X-fast trie.
3. Add the representative of the new tree into X-fast trie.
4. If the newly-formed tree contains more than  $2 \log U$  elements (where  $U$  is the maximum value of the domain), then remove its representative from the X-fast trie. Split the binary search tree into two and add the representatives from each of the binary search trees into the X-fast trie.

### Check Your Progress – 2

1. The child nodes of a node in a Trie are associated with a \_\_\_\_\_
2. The average time complexity for search, insert and delete operations for trie data structure is \_\_\_\_\_
3. The binary tries use \_\_\_\_\_ as keys that can be used to represent integer
4. X-fast tries are bitwise tries that store the integers from a \_\_\_\_\_
5. Each level of the x-fast trie is implemented as \_\_\_\_\_
6. Y-fast tries are the improved version of \_\_\_\_\_

---

## 11.7 SUMMARY

---

In this unit, we started discussing the key aspects of binary search tree. Binary search tree is a sorted tree that stores the keys in ordered way. The scapegoat tries are self-balancing binary search tree. When the weight of the left nodes matches the weight of the right nodes then we say that the binary search tree is balanced. To insert the data we find the scapegoat node in scapegoat trees and balance the tree around it. Tries are tree-based data structure that are used to search for string-based keys in a given set. The average time complexity for search, insert and delete operations for trie data structure is  $O(n)$ . The binary tries use bits as keys that can be used to represent integer. X-fast tries are bitwise tries that store the integers from a bounded domain.

---

## 11.8 SOLUTIONS/ANSWERS

---

### Check Your Progress – 1

1. Self-balancing
2. scapegoat
3.  $O(\log n)$
4. Lesser
5.  $O(n)$

### Check Your Progress – 2

1. common prefix
2.  $O(n)$
3. bits
4. bounded domain
5. hash table

---

## 11.9 FURTHER READINGS

---

Horowitz, Ellis, Sartaj Sahni, and Susan Anderson-Freed. *Fundamentals of data structures*. Vol. 20. Potomac, MD: Computer science press, 1976.

Cormen, Thomas H., et al. *Introduction to algorithms*. MIT press, 2022.

Lafore, Robert. *Data structures and algorithms in Java*. Sams publishing, 2017.

Karumanchi, Narasimha. *Data structures and algorithms made easy: data structure and algorithmic puzzles*. Narasimha Karumanchi, 2011.

West, Douglas Brent. *Introduction to graph theory*. Vol. 2. Upper Saddle River: Prentice hall, 2001.

